



Univerzitet u Kragujevcu



Prirodno-matematički fakultet



Institut za matematiku i informatiku

Projekat 1

Entropija, Shannon-Fano, Huffman, LZ77, LZW

Kompresija i zaštita podataka

Student:

Aleksa Radivojević, 76/2021

Mentor:

Marko Petković, profesor

Sadržaj

Struktura projekta	3
Entropija	3
Implementacija entropije	3
Shannon-Fano	4
Implementacija Shannon-Fano	5
Huffmanov algoritam	7
Implementacija Huffmana	8
LZ77 algoritam	10
Implementacija LZ77	11
LZW	13
Implementacija LZW	13

Struktura projekta

Projekat je realizovan u programskom jeziku **C++**, dok su struktura koda i sistem izgradnje organizovani su po sledećim principima:

- **Modularna struktura koda (Header / Source separacija):** Kod je podeljen na zaglavlja (.h) u headers direktorijumu i izvornim datotekama (.cpp) u cpp direktorijumu. Ova podela obezbeđuje čist interfejs, sprečava konflikte pri višestrukome uključivanju (pomoću *include guard*-ova) i omogućava brže kompajliranje jer se menja samo onaj deo koda koji je izmenjen.
- **Upotreba CMake sistema za izgradnju:** Za upravljanje prevođenjem projekta koristi se **CMake** (CMakeLists.txt), razlog za upotrebu **Cmake-a** jeste taj što omogućava portabilnost projekta na bilo kom OS-u, a da pritom omogućava lakše kompajliranje, odnosno ponovo se kompajlira samo onaj deo koda koji je izmenjen.

Entropija

Entropija predstavlja meru neodređenosti, odnosno prosečne količine bitova potrebnih kako bi se neki simbol, tj. podatak predstavio. Sama funkcija entropije daje vrednost između 0 i 8 bitova po bajtu, zato što bajt može poprimiti 256 različitih stanja. Entropija je niska kada se u skupu podataka nalazi mnogo ponavljanja, odnosno kada su podaci u skupu predvidivi. Entropija kao mera predstavlja osnovu kompresije podataka bez gubitaka zato što nam ona teorijski pokazuje koliko nam je minimalno bitova po simbolu potrebno da bismo sačuvali podatke, kao i to koliko će sama kompresija biti efektivno sprovedena.

Formula entropije (data u postavci projekta):

$$H(p) = -p_0 \log_2 p_0 - p_1 \log_2 p_1 - \dots - p_{255} \log_2 p_{255} \Leftrightarrow H(p) = - \sum_{i=0}^{N_i} p_i \log_2 p_i$$

Implementacija entropije

```

#include "../headers/entropy.h"
#include <fstream>
#include <cmath>
#include <vector>
#include <iostream>

pair<double, vector<double>> bajtEntropija(const string& file) {
    ifstream fajl(file, ios::binary);
    if (!fajl.is_open()) {
        cerr << "Greska pri otvaranju fajla: " << file << endl;
        return {0.0, vector<double>(256, 0.0)};
    }

    vector<uint8_t> podaci((istreambuf_iterator<char>(fajl), istreambuf_iterator<char>()));
    fajl.close();

    size_t N = podaci.size();
    if (N == 0) return {0.0, vector<double>(256, 0.0)};

    vector<long long> Ni(256, 0);
    for (uint8_t bajt : podaci) {
        Ni[bajt]++;
    }

    double entropija = 0.0;
    vector<double> pi(256, 0.0);

    for (int i = 0; i < 256; ++i) {
        if (Ni[i] == 0) continue;
        pi[i] = static_cast<double>(Ni[i]) / N;

        entropija -= (pi[i] * log2(pi[i]));
    }

    return {entropija, pi};
}

```

Funkcija pored vrednosti entropije vraća i niz verovatnoća koji će se koristiti u Shannon-Fano kodiranju.

Shannon-Fano

Shannon-Fano algoritam je jedan od najranijih postupaka za bezgubitnu kompresiju podataka, koji su nezavisno razvili Klod Šenon i Robert Fano 1949. godine. Glavna ideja algoritma je da se simbolima koji se češće pojavljuju u fajlu dodele kraći binarni kodovi, dok se ređim simbolima dodeljuju duži kodovi, čime se smanjuje ukupna veličina fajla. Algoritam primenjuje koncept kodiranja varijabilnom dužinom kroz odozgo-nadole pristup kreiranju prefiksnih kodova. Proces započinje analizom ulaznog fajla u kojoj se prebrojavaju svi prisutni bajtovi i izračunava njihova verovatnoća pojavljivanja. Nakon toga, svi simboli se sortiraju u opadajućem redosledu, tako da najčešći bajtovi budu na početku niza, a najređi na kraju. Ključni korak algoritma sastoji se u rekursivnom deljenju sortiranog niza na dve grupe. Podela se vrši na mestu gde je zbir verovatnoća simbola u levoj polovini što je moguće bliži zbiru verovatnoća u desnoj polovini. Elementima u prvoj grupi na postojeći kod dodaje se bit 0, dok se elementima u drugoj grupi dodaje bit 1. Ovaj postupak deljenja podgrupa i dodeljivanja bitova rekursivno se ponavlja sve dok se u svakoj podgrupi ne nađe samo po jedan simbol. Na taj način česti simboli dobijaju kraće binarne zapise, dok ređi dobijaju duže, čime se ostvaruje kompresija bez gubitka podataka.

Implementacija Shannon-Fano

Implementacija Shannon-Fano kompresije i dekompresije oslanja se na tri glavne funkcije unutar samog modul-fajla, kao i na eksterne pomoćne funkcije za statističku analizu i sortiranje.

Funkcija **shannonFano** predstavlja rekurzivni algoritam koji generiše binarne kodove. Na samom početku proverava bazni slučaj rekurzije, gde ukoliko je ostao samo jedan simbol u grupi vraća prazan kod. Nakon toga poziva eksternu funkciju **insertionSortOpadajuće** koja sortira bajtove po verovatnoći od najveće ka najmanjoj. Zatim izračunava ukupnu sumu verovatnoća u grupi i prolazi kroz niz sve dok tempSuma ne pređe polovinu ukupne verovatnoće. Na tom mestu deli niz na levu i desnu grupu te rekurzivno poziva sama sebe. Rezultatima leve grupe sa leve strane dodaje bit "0", a desnoj "1", i spaja ih u jedinstvenu mapu kodova.

```
unordered_map<uint8_t, string> shannonFano(vector<BajtVerovatnoca> bajtoviVerovatnoce){
    if (bajtoviVerovatnoce.size() == 1) {
        return {{bajtoviVerovatnoce[0].first, ""}};
    }

    insertionSortOpadajuće(bajtoviVerovatnoce);

    double ukupnaVerovatnocaGrupe = 0;
    for (const auto& item : bajtoviVerovatnoce) {
        ukupnaVerovatnocaGrupe += item.second;
    }

    double tempSuma = 0;
    size_t i = 0;
    for (i = 0; i < bajtoviVerovatnoce.size(); ++i) {
        tempSuma += bajtoviVerovatnoce[i].second;
        if (tempSuma >= ukupnaVerovatnocaGrupe / 2.0) {
            break;
        }
    }

    vector<BajtVerovatnoca> levaGrupa(bajtoviVerovatnoce.begin(), bajtoviVerovatnoce.begin() + i + 1);
    vector<BajtVerovatnoca> desnaGrupa(bajtoviVerovatnoce.begin() + i + 1, bajtoviVerovatnoce.end());

    auto leviKod = shannonFano(levaGrupa);
    auto desniKod = shannonFano(desnaGrupa);

    unordered_map<uint8_t, string> codes;
    for (const auto& pair : leviKod) {
        codes[pair.first] = "0" + pair.second;
    }

    for (const auto& pair : desniKod) {
        codes[pair.first] = "1" + pair.second;
    }

    return codes;
}
```

Funkcija **code_sha_fan** vodi proces kompresije ulaznog fajla. Prvo poziva eksternu funkciju **bajtEntropija** koja vraća verovatnoće pojavljivanja bajtova, a zatim eksternu funkciju **listaSimbolaVerovatnoca** kako bi te podatke pretvorila u podložan format za sortiranje. Potom poziva **shannonFano** funkciju za generisanje mape kodova. Učitava sirove bajtove iz ulaznog fajla i menja ih odgovarajućim binarnim kodovima u tekstualnom baferu. Čim bafer dostigne dužinu od 8 ili više bitova, prvih 8 bitova konvertuje

u `uint8_t` bajt i upisuje ga u binarni izlazni fajl. Ukoliko na kraju ostane nepun bajt, dopunjava ga nulama, upisuje u fajl i pamti broj stvarno validnih bitova u tom poslednjem bajtu, vraćajući par koji čine generisana mapa i broj validnih bitova.

```
pair<unordered_map<uint8_t, string>, int> code_sha_fan(const string& input, const string& output) {
    auto [entropija, pi] = bajtEntropija(input);
    auto bajtoviVerovatnoce = listaSimbolaVerovatnoce(pi);
    auto kod = shannonFano(bajtoviVerovatnoce);

    ifstream fajlUlaz(input, ios::binary);
    ofstream fajlIzlaz(output, ios::binary);

    vector<uint8_t> podaci((istreambuf_iterator<char>(fajlUlaz)), istreambuf_iterator<char>());
    fajlUlaz.close();

    string kodiraniPodaciBafer = "";
    for (uint8_t bajt : podaci) {
        kodiraniPodaciBafer += kod[bajt];

        while (kodiraniPodaciBafer.length() >= 8) {
            uint8_t bajtZaUpis = static_cast<uint8_t>(stoi(kodiraniPodaciBafer.substr(0, 8), nullptr, 2));
            fajlIzlaz.write(reinterpret_cast<char*>(&bajtZaUpis), 1);
            kodiraniPodaciBafer = kodiraniPodaciBafer.substr(8);
        }
    }

    int valid = 8;
    if (!kodiraniPodaciBafer.empty()) {
        valid = kodiraniPodaciBafer.length();
        kodiraniPodaciBafer.append(8 - valid, '0');
        uint8_t bajtZaUpis = static_cast<uint8_t>(stoi(kodiraniPodaciBafer, nullptr, 2));
        fajlIzlaz.write(reinterpret_cast<char*>(&bajtZaUpis), 1);
    }

    fajlIzlaz.close();
    return {kod, valid};
}
```

Funkcija **decodeSH** vrši rekonstrukciju originalnog fajla. Najpre pravi inverznu mapu, odnosno rečnik koji povezuje binarni string sa odgovarajućim bajtom radi brže pretrage. Zatim učitava sve kompresovane bajtove iz binarnog fajla i pomoću `bitset<8>` strukture konvertuje ih u jedan dugi niz bitova, vodeći računa da se iz poslednjeg bajta uzme samo onoliko bitova koliko je zabeleženo u promenljivoj `valid`. Prolaskom kroz dobijeni niz bitova postepeno spaja bit po bit u promenljivu uzorak. Čim stvoreni uzorak odgovara nekom kodu iz rečnika, upisuje prepoznati bajt u izlazni niz i resetuje uzorak, a na samom kraju sve dekodirane bajtove binarno upisuje u izlazni fajl.

```

void decodeSH(const string& coded, const string& decoded,
const unordered_map<uint8_t, string>& codes, int valid){
    unordered_map<string, uint8_t> rechnik;
    for (const auto& [bajt, kod] : codes) {
        rechnik[kod] = bajt;
    }

    ifstream fajl(coded, ios::binary);
    vector<uint8_t> bajtovi((istreambuf_iterator<char>(fajl) ), istreambuf_iterator<char>());
    fajl.close();

    string bitovi = "";
    for (size_t i = 0; i < bajtovi.size(); ++i) {
        string bBits = bitset<8>(bajtovi[i]).to_string();
        if (i == bajtovi.size() - 1) {
            bitovi += bBits.substr(0, valid);
        } else {
            bitovi += bBits;
        }
    }

    string uzorak = "";
    vector<uint8_t> dekodiraniBajtovi;

    for (char bit : bitovi) {
        uzorak += bit;
        if (rechnik.find(uzorak) != rechnik.end()) {
            dekodiraniBajtovi.push_back(rechnik[uzorak]);
            uzorak = "";
        }
    }

    ofstream fajlIzlaz(decoded, ios::binary);
    fajlIzlaz.write(reinterpret_cast<char*>(dekodiraniBajtovi.data()), dekodiraniBajtovi.size());
    fajlIzlaz.close();
}

```

Huffmanov algoritam

Huffmanov algoritam primenjuje koncept kodiranja varijabilnom dužinom kroz odozdo-nagore pristup izgradnji binarnog stabla, čime se garantuje matematički optimalan prefiksni kod za zadatu raspodelu simbola. Proces započinje prebrojavanjem učestanosti svih bajtova prisutnih u ulaznom fajlu. Na osnovu izračunatih frekvencija, svaki simbol se pretvara u pojedinačni čvor, a svi čvorovi se smeštaju u red sa prioritetom koji uvek na vrhu drži elemente sa najmanjom frekvencijom. Glavni deo algoritma odvija se kroz sukcesivno spajanje čvorova. Iz reda sa prioritetom sukcesivno se izvlače dva čvora sa najmanjim frekvencijama i spajaju u novi roditeljski čvor čija je frekvencija jednaka zbiru njihovih pojedinačnih frekvencija. Izvučeni čvorovi postaju levo i desno dete novokreiranog čvora, dok se roditeljski čvor vraća nazad u red sa prioritetom. Ovaj postupak spajanja najređih elemenata ponavlja se sve dok u redu ne ostane samo jedan korenski čvor koji ujedinjuje celokupno Huffmanovo stablo.

Generisanje samih kodova vrši se prolaskom kroz stablo od korena ka listovima, pri čemu se kretanju ulevo dodeljuje bit 0, a kretanju udesno bit 1. Putanja od korenskog čvora do pojedinačnog lista predstavlja konačni binarni kod tog bajta. Na taj način česti simboli, koji se nalaze blizu vrha stabla, dobijaju izuzetno kratke kodove, dok ređi simboli dobijaju duže zapise. S obzirom na to da su svi generisani kodovi prefiksni, dekodiranje kompresovanog niza vrši se jednoznačno i bez potrebe za graničnicima.

Implementacija Huffmana

Pomoćna rekurzivna funkcija **huffCodes** služi za generisanje prefiksnih binarnih kodova obilaskom formiranog Huffmanovog stabla. Prolazeći kroz stablo od korenskog čvora ka listovima, pri svakom koraku ulevo prefiksu dodaje bit "0", a pri koraku udesno bit "1". Kada funkcija naiđe na list, odnosno čvor koji predstavlja konkretan bajt, u mapu kodovi upisuje stvoreni binarni niz za taj bajt, pri čemu rukuje i specijalnim slučajem kada ulaz sadrži samo jedan jedinstveni simbol.

```
void huffCodes(shared_ptr<Node> root, const string& prefiks, unordered_map<uint8_t, string>& kodovi) {
    if (!root) return;

    if (root->isLeaf) {
        kodovi[root->bajt] = prefiks.empty() ? "0" : prefiks;
    } else {
        huffCodes(root->levo, prefiks + "0", kodovi);
        huffCodes(root->desno, prefiks + "1", kodovi);
    }
}
```

Funkcija **codeHuffman** upravlja celokupnim procesom izgradnje stabla i kompresije ulazne datoteke. Najpre poziva eksterne funkcije **bajtEntropija** i **listaSimbolaVerovatnoca** kako bi pribavila listu bajtova i njihovih verovatnoća, a zatim od svakog simbola kreira pojedinačni čvor obmotan pametnim pokazivačem `std::shared_ptr`. U glavnoj petlji sukcesivno sortira niz čvorova u opadajućem redosledu po verovatnoći, izvlaci dva čvora sa najmanjom učestalošću sa kraja niza, spaja ih u novi roditeljski čvor i vraća ga nazad u niz sve dok ne ostane samo korensko stablo. Nakon generisanja koda pozivom funkcije **huffCodes**, učitava bajtove iz ulazne datoteke, konvertuje ih u odgovarajući binarni niz u tekstualnom baferu i svakih formiranih 8 bitova upisuje kao jedan bajt u izlazni binarni fajl. Ukoliko na kraju ostane nepun bajt, dopunjava ga nulama do punog bajta i vraća par koji sadrži mapu kodova i broj stvarno validnih bitova u poslednjem bajtu.


```

pair<unordered_map<uint8_t, string>, int> codeHuffman(const string& input, const string& output)
{
    auto [entropija, pi] = bajtEntropija(input);
    auto bajtoviVerovatnoce = listaSimbolaVerovatnoca(pi);

    vector<shared_ptr<Node>> cvorovi;
    for (const auto& bv : bajtoviVerovatnoce) {
        cvorovi.push_back(make_shared<Node>(bv.first, bv.second));
    }

    if (cvorovi.empty()) return {{}, 8};

    while (cvorovi.size() > 1) {
        sort(cvorovi.begin(), cvorovi.end(), [](const shared_ptr<Node>& a, const shared_ptr<Node>& b) {
            return a->verovatnoca > b->verovatnoca;
        });

        auto c1 = cvorovi.back(); cvorovi.pop_back();
        auto c2 = cvorovi.back(); cvorovi.pop_back();

        auto noviCvor = make_shared<Node>(c1, c2);
        cvorovi.push_back(noviCvor);
    }

    unordered_map<uint8_t, string> kodovi;
    huffCodes(cvorovi[0], "", kodovi);

    ifstream fajlUlaz(input, ios::binary);
    ofstream fajlIzlaz(output, ios::binary);

    vector<uint8_t> podaci((istreambuf_iterator<char>(fajlUlaz)), istreambuf_iterator<char>());
    fajlUlaz.close();

    string kodiraniPodaciBafer = "";
    for (uint8_t bajt : podaci) {
        kodiraniPodaciBafer += kodovi[bajt];

        while (kodiraniPodaciBafer.length() >= 8) {
            uint8_t bajtZaUpis = static_cast<uint8_t>(stoi(kodiraniPodaciBafer.substr(0, 8), nullptr, 2));
            fajlIzlaz.write(reinterpret_cast<char*>(&bajtZaUpis), 1);
            kodiraniPodaciBafer = kodiraniPodaciBafer.substr(8);
        }

        int validniBitovi = 8;
        if (!kodiraniPodaciBafer.empty()) {
            validniBitovi = kodiraniPodaciBafer.length();
            kodiraniPodaciBafer.append(8 - validniBitovi, '0');
            uint8_t bajtZaUpis = static_cast<uint8_t>(stoi(kodiraniPodaciBafer, nullptr, 2));
            fajlIzlaz.write(reinterpret_cast<char*>(&bajtZaUpis), 1);
        }

        fajlIzlaz.close();
        return {kodovi, validniBitovi};
    }
}

```

Funkcija **decodeHuffman** vrši proces dekompresije i rekonstruiše originalne podatke na osnovu kompresovanog binarnog fajla i predate mape kodova. Prvo pravi inverznu mapu obrnutiKodovi koja omogućava brzu pretragu bajta na osnovu njegovog binarnog koda. Zatim učitava kompresovane bajtove i pomoću bitset<8> strukture pretvara ih u jedinstveni string bitova, vodeći računa da se iz poslednjeg bajta pročita samo onoliko bitova koliko definiše parametar validniBitovi. Prolaskom kroz dobijeni niz bitova postepeno formira uzorak i čim ga pronade u inverznoj mapi, upisuje odgovarajući dekodirani bajt u rezultat i resetuje uzorak, nakon čega sve rekonstruisane bajtove binarno upisuje u izlaznu datoteku.

```

void decodeHuffman(const string& kodiraniFajl, const string& dekodiraniFajl, const unordered_map<uint8_t, string>& kodovi, int validniBitovi) {
    unordered_map<string, uint8_t> obrnutiKodovi;
    for (const auto& [bajt, kod] : kodovi) {
        obrnutiKodovi[kod] = bajt;
    }

    ifstream fajl(kodiraniFajl, ios::binary);
    if (!fajl) return;

    vector<uint8_t> bajtovi((istreambuf_iterator<char>(fajl)), istreambuf_iterator<char>());
    fajl.close();

    if (bajtovi.empty()) return;

    string bitovi = "";
    for (size_t i = 0; i < bajtovi.size(); ++i) {
        string bBits = bitset<8>(bajtovi[i]).to_string();
        if (i == bajtovi.size() - 1) {
            bitovi += bBits.substr(0, validniBitovi);
        } else {
            bitovi += bBits;
        }
    }

    vector<uint8_t> dekodiraniBajtovi;
    string uzorak = "";
    for (char bit : bitovi) {
        uzorak += bit;
        if (obrnutiKodovi.find(uzorak) != obrnutiKodovi.end()) {
            dekodiraniBajtovi.push_back(obrnutiKodovi[uzorak]);
            uzorak = "";
        }
    }

    ofstream fajlIzlaz(dekodiraniFajl, ios::binary);
    fajlIzlaz.write(reinterpret_cast<char*>(dekodiraniBajtovi.data()), dekodiraniBajtovi.size());
    fajlIzlaz.close();
}

```

LZ77 algoritam

LZ77 algoritam je kompresioni postupak zasnovan na rečniku koji je prvi put predstavljen od strane Abrahama Lempela i Džejkoba Ziva 1977. godine. Za razliku od algoritama koji se oslanjaju na statističku verovatnoću pojedinačnih simbola, LZ77 kompresuje podatke uočavanjem i zamenjivanjem ponovljenih sekvenci tekstova ili bajtova. Algoritam funkcioniše tako što koristi takozvani klizni prozor koji se pomera kroz ulazne podatke od početka ka kraju. Ovaj prozor podeljen je na dva dela: bafer pretrage koji sadrži već viđene i obrađene podatke, i bafer unapred koji sadrži bajtove koji tek treba da budu kompresovani.

Proces kompresije odvija se tako što algoritam pretražuje bafer pretrage kako bi pronašao najduže poklapanje sa bajtovima koji se trenutno nalaze u baferu unapred. Kada se poklapanje pronađe, sekvenca se ne zapisuje kao sirovi bajtovi, već se zamenjuje strukturom poznatom kao LZ77 trojka. Ova trojka se sastoji od pomaka koji označava koliko se bajtova unazad u baferu pretrage nalazi početak poklapanja, dužine koja označava koliko se uzastopnih bajtova poklapa, i sledećeg karaktera koji predstavlja prvi bajt u baferu unapred koji se nije poklopio. Ukoliko nikakvo poklapanje ne postoji, algoritam izmituje trojku sa pomakom i dužinom nula, uz sam trenutni bajt. Nakon svake kreirane trojke klizni prozor se pomera unapred za dužinu poklapanja uvećanu za jedan, čime se proces ponavlja sve do kraja ulaznih podataka i ostvaruje znatna ušteda u prostoru kod fajlova sa učestalim ponavljanjima.

Implementacija LZ77

Sama logika kompresije realizovana je u funkciji **lz77**, koja ulazni niz bajtova analizira pomoću kliznog prozora definisane veličine. Funkcija prolazi kroz podatke i u svakom koraku pretražuje prethodno obrađene bajtove u opsegu pretrage kako bi pronašla najduže poklapanje sa bajtovima u baferu unapred. Izračunava se najduži pomak i dužina sekvence, nakon čega se formira trojka tipa **LZ77Triple** (definisana kao **std::tuple<uint16_t, uint8_t, uint8_t>**) koja sadrži pomak unazad, dužinu poklapanja i prvi sledeći nepoklapajući bajt. Indeks pretrage se potom uvećava za dužinu poklapanja plus jedan, a generisani niz trojki vraća kao rezultat.

```
vector<LZ77Triple> lz77(const vector<uint8_t>& podaci, size_t velicinaProzor, size_t bajtoviUnapred) {
    vector<LZ77Triple> kod;
    size_t n = podaci.size();
    if (n == 0) return kod;

    size_t i = 0;
    while (i < n) {
        if (i == n - 1) {
            kod.push_back({0, 0, podaci[i]});
            break;
        }

        size_t maxDuzina = min(bajtoviUnapred, n - i - 1);
        size_t pProzor = (i > velicinaProzor) ? (i - velicinaProzor) : 0;

        uint16_t najPomak = 0;
        uint8_t najDuzina = 0;

        for (size_t p = pProzor; p < i; ++p) {
            size_t l = 0;
            while (l < maxDuzina && podaci[p + l] == podaci[i + l]) {
                l++;
            }
            if (l > najDuzina) {
                najDuzina = static_cast<uint8_t>(l);
                najPomak = static_cast<uint16_t>(i - p);
                if (najDuzina == maxDuzina) break;
            }
        }

        size_t sledeciI = i + najDuzina;
        uint8_t sledeci = (sledeciI < n) ? podaci[sledeciI] : 0;

        kod.push_back({najPomak, najDuzina, sledeci});
        i += najDuzina + 1;
    }

    return kod;
}
```

Glavna funkcija za kompresiju **codeLZ77** učitava sirove bajtove iz ulazne datoteke i prosleđuje ih funkciji **lz77**. Pre upisa samih trojki u izlazni binarni fajl, funkcija formira zaglavlje zapisivanjem ukupnog broja trojki kao 32-bitnog neoznačenog celog broja kroz 4 bajta u *big-endian* formatu. Nakon zaglavlja, kroz petlju se vrši serijalizacija svake trojke iz vektora tako što se 16-bitni pomak deli i zapisuje kroz 2 bajta, dok se dužina i sledeći bajt zapisuju pojedinačno po 1 bajt, čime svaka trojka zauzima tačno 4 bajta u binarnom fajlu.

```

vector<LZ77Triple> codeLZ77(const string& input, const string& output) {
    ifstream fajlUlaz(input, ios::binary);
    vector<uint8_t> podaci((istreambuf_iterator<char>(fajlUlaz)), istreambuf_iterator<char>());
    fajlUlaz.close();

    auto kodovi = lz77(podaci);

    ofstream fajlIzlaz(output, ios::binary);
    uint32_t brojKodova = static_cast<uint32_t>(kodovi.size());

    uint8_t buffer4[4] = {
        static_cast<uint8_t>((brojKodova >> 24) & 0xFF),
        static_cast<uint8_t>((brojKodova >> 16) & 0xFF),
        static_cast<uint8_t>((brojKodova >> 8) & 0xFF),
        static_cast<uint8_t>(brojKodova & 0xFF)
    };
    fajlIzlaz.write(reinterpret_cast<char*>(buffer4), 4);

    for (const auto& [pomak, duzina, sledeci] : kodovi) {
        uint8_t buffer2[2] = {
            static_cast<uint8_t>((pomak >> 8) & 0xFF),
            static_cast<uint8_t>(pomak & 0xFF)
        };
        fajlIzlaz.write(reinterpret_cast<char*>(buffer2), 2);
        fajlIzlaz.write(reinterpret_cast<const char*>(&duzina), 1);
        fajlIzlaz.write(reinterpret_cast<const char*>(&sledeci), 1);
    }

    fajlIzlaz.close();
    return kodovi;
}

```

Funkcija **decodeLZ77** rekonstruiše originalni fajl čitanjem kompresovanog binarnog sadržaja. Najpre iz prva 4 bajta rekonstruiše zaglavlje, odnosno tačan broj trojki koje treba pročitati. Zatim u petlji čita sukcesivne blokove od po 4 bajta, izdvajajući pomak, dužinu i nespareni simbol. Ukoliko je pomak veći od nule, funkcija iz već dekodiranog niza bajtova na osnovu izračunate pozicije prepisuje odgovarajući broj bajtova, nakon čega uvek dodaje i pročitani nespareni simbol, čime u potpunosti obnavlja originalni niz bajtova i upisuje ga u izlazni fajl.

```

void decodeLZ77(const string& kodiraniFajl, const string& dekodiraniFajl)
{
    std::ifstream fajlUlaz(kodiraniFajl, std::ios::binary);
    std::vector<uint8_t> podaci((std::istreambuf_iterator<char>){fajlUlaz}), std::istreambuf_iterator<char>());
    fajlUlaz.close();

    std::ofstream fajlIzlaz(dekodiraniFajl, std::ios::binary);
    if (podaci.size() < 4) return;

    size_t pozicija = 0;
    uint32_t brojKodova = (podaci[0] << 24) | (podaci[1] << 16) | (podaci[2] << 8) | podaci[3];
    pozicija += 4;

    std::vector<uint8_t> dekodiraniBajtovi;

    for (uint32_t z = 0; z < brojKodova; ++z) {
        if (pozicija + 4 > podaci.size()) break;

        uint16_t pomak = (podaci[pozicija] << 8) | podaci[pozicija + 1];
        uint8_t duzina = podaci[pozicija + 2];
        uint8_t simbol = podaci[pozicija + 3];
        pozicija += 4;

        if (pomak > 0 && dekodiraniBajtovi.size() >= pomak) {
            for (uint8_t _i = 0; _i < duzina; ++_i) {
                uint8_t bajt = dekodiraniBajtovi[dekodiraniBajtovi.size() - pomak];
                dekodiraniBajtovi.push_back(bajt);
            }
        }
        dekodiraniBajtovi.push_back(simbol);
    }

    fajlIzlaz.write(reinterpret_cast<char*>(dekodiraniBajtovi.data()), dekodiraniBajtovi.size());
    fajlIzlaz.close();
}

```

LZW

LZW (Lempel-Ziv-Welch) je algoritam za bezgubitnu kompresiju zasnovan na rečniku, koji dinamički gradi tabele sekvenci u hodu, bez potrebe za prethodnom analizom podataka ili slanjem rečnika u izlazni fajl. Algoritam počinje sa inicijalizovanim rečnikom od 256 osnovnih bajt-vrednosti. Tokom kompresije, čita bajt po bajt i spaja ih u nizove; čim naiđe na sekvencu koja se ne nalazi u rečniku, u izlaz upisuje indeks prethodno poznatog dela niza, dok novu sekvencu dodaje u rečnik pod sledećim slobodnim indeksom.

Dekompresija je simetrična i brza. Dekoder kreće od istog osnovnog rečnika i, prevođenjem pročitanih indeksa nazad u sekvence, samostalno rekonstruiše identičan rečnik koji je koder gradio tokom kompresije. Zbog efikasnosti i prilagodljivosti u hodu, LZW je našao široku primenu u formatima kao što su GIF, TIFF i PDF.

Implementacija LZW

Funkcija **codeLZW** najpre inicijalizuje dinamički rečnik sa osnovnih 256 bajt-vrednosti, spajajući pojedinačne bajtove u nizove radi uočavanja ponavljanja sekvenci. Tokom prolaska kroz ulazne podatke, spaja trenutni bajt sa prethodno formiranim nizom i proverava postojanje u rečniku. Kada naiđe na dosad

nepostojeću sekvencu, u listu kodiranih podataka upisuje indeks prethodnog poznatog niza, dok novu sekvencu dodaje u rečnik pod sledećim slobodnim indeksom, ograničavajući maksimalnu veličinu rečnika na 4096 ulaza. Generisane celobrojne indekse potom pretvara u 12-bitne binarne zapise pomoću `bitset<12>`, pakuje ih u bajtove od po 8 bitova i upisuje u izlazni binarni fajl, pri čemu za eventualno nepun poslednji bajt dopunjava nule i vraća par koji sadrži rečnik i broj validnih bitova.

```
pair<map<vector<uint8_t>, int>, int> codeLZW(const string& input, const string& output) {  
    ifstream fajlUlaz(input, ios::binary);  
    vector<uint8_t> podaci((istreambuf_iterator<char>(fajlUlaz)), istreambuf_iterator<char>());  
    fajlUlaz.close();  
  
    map<vector<uint8_t>, int> rečnik;  
    for (int i = 0; i < 256; ++i) {  
        rečnik[{static_cast<uint8_t>(i)}] = i;  
    }  
    int velicinaRečnika = 256;  
    int velicinaRečnikaMax = 4096;  
  
    vector<int> kodiraniPodaci;  
    vector<uint8_t> bajtoviNiz;  
  
    for (uint8_t bajt : podaci) {  
        vector<uint8_t> temp = bajtoviNiz;  
        temp.push_back(bajt);  
  
        if (rečnik.find(temp) != rečnik.end()) {  
            bajtoviNiz = temp;  
        } else {  
            if (!bajtoviNiz.empty()) {  
                kodiraniPodaci.push_back(rečnik[bajtoviNiz]);  
            }  
            if (velicinaRečnika < velicinaRečnikaMax) {  
                rečnik[temp] = velicinaRečnika++;  
            }  
            bajtoviNiz = {bajt};  
        }  
    }  
    if (!bajtoviNiz.empty()) {  
        kodiraniPodaci.push_back(rečnik[bajtoviNiz]);  
    }  
  
    ofstream fajlIzlaz(output, ios::binary);  
    string kodiraniPodaciBafer = "";
```



```

for (int kod : kodiraniPodaci) {
    kodiraniPodaciBafer += bitset<12>(kod).to_string();
    while (kodiraniPodaciBafer.length() >= 8) {
        uint8_t bajt = static_cast<uint8_t>(stoi(kodiraniPodaciBafer.substr(0, 8), nullptr, 2));
        fajlIzlaz.write(reinterpret_cast<char*>(&bajt), 1);
        kodiraniPodaciBafer = kodiraniPodaciBafer.substr(8);
    }
}

int validniBitovi = 8;
if (!kodiraniPodaciBafer.empty()) {
    validniBitovi = kodiraniPodaciBafer.length();
    kodiraniPodaciBafer.append(8 - validniBitovi, '0');
    uint8_t bajt = static_cast<uint8_t>(stoi(kodiraniPodaciBafer, nullptr, 2));
    fajlIzlaz.write(reinterpret_cast<char*>(&bajt), 1);
}

fajlIzlaz.close();
return {rechnik, validniBitovi};
}

```

Funkcija **decodeLZW** vrši dekompresiju prevođenjem izlaznih 12-bitnih kodova nazad u originalne sekvence bajtova. Najpre kreira inverznu mapu u kojoj indksi menjaju mesta sa odgovarajućim sekvencama bajtova radi direktnog i brzog pristupa. Nakon učitavanja kompresovane datoteke, pretvara pročitane bajtove u kontinualni niz bitova pomoću `bitset<8>`, uzimajući u obzir tačan broj validnih bitova u poslednjem bajtu. Zatim u petlji izdvaja sukcesivne blokove od po 12 bitova, konvertuje ih u celobrojne kôdne indekse i na osnovu rečnika rekonstruiše originalne sekvence bajtova, koje na kraju spaja i zapisuje u izlazni fajl.

```

void decodeLZW(const string& kodiraniFajl, const string& dekodiraniFajl, const map<vector<uint8_t>, int>& kodovi, int validniBitovi) {
    unordered_map<int, vector<uint8_t>> rechnik;
    for (const auto& [bajtovi, kod] : kodovi) {
        rechnik[kod] = bajtovi;
    }

    ifstream fajl(kodiraniFajl, ios::binary);
    vector<uint8_t> bajtovi((istreambuf_iterator<char>(fajl), istreambuf_iterator<char>()), istreambuf_iterator<char>());
    fajl.close();

    string bitovi = "";
    for (size_t i = 0; i < bajtovi.size(); ++i) {
        string bBits = bitset<8>(bajtovi[i]).to_string();
        if (i == bajtovi.size() - 1) {
            bitovi += bBits.substr(0, validniBitovi);
        } else {
            bitovi += bBits;
        }
    }

    vector<uint8_t> dekodiraniBajtovi;
    while (bitovi.length() >= 12) {
        string uzorak = bitovi.substr(0, 12);
        int kod = stoi(uzorak, nullptr, 2);
        if (rechnik.find(kod) != rechnik.end()) {
            const auto& val = rechnik[kod];
            dekodiraniBajtovi.insert(dekodiraniBajtovi.end(), val.begin(), val.end());
        }
        bitovi = bitovi.substr(12);
    }

    ofstream fajlIzlaz(dekodiraniFajl, ios::binary);
    fajlIzlaz.write(reinterpret_cast<char*>(dekodiraniBajtovi.data()), dekodiraniBajtovi.size());
    fajlIzlaz.close();
}

```

Opšte funkcije

Kako bi se implementacije pojedinih algoritama, odnosno Shannon-Fano i Huffman-a uspešno izvršile, a kako ne bi dolazilo do konstantnog ponavljanja koda u samim implementacijama algoritama, razvijena je posebna celina zaglavljiva „functions.h“ koja sadrži osnovne definicije za funkcije **listaSimbolaVerovatnoca** i **insertionSortOpadajuće** kao i strukturu **BajtVerovatnoca** koja povezuje konkretnu vrednost bajta sa njegovom izračunatom vrednošću pojavljivanja.

- **listaSimbolaVerovatnoca** uzima vektor verovatnoća svih 256 bajtova dobijen iz funkcije **bajtEntropija** i filtrira ga tako što izdvaja samo one bajtove koji se zapravo pojavljuju u fajlu (čija je verovatnoća veća od nule), pakujući ih u vektor **BajtVerovatnoca** parova.
- **insertionSortOpadajuće** primenjuje algoritam sortiranja umetanjem (Insertion Sort) nad vektorom **BajtVerovatnoca** kako bi sortirao simboličke parove u opadajućem redosledu prema njihovoj verovatnoći.

```
vector<BajtVerovatnoca> listaSimbolaVerovatnoca(const vector<double>& pi) {
    vector<BajtVerovatnoca> bajtoviVerovatnoce;
    for (int i = 0; i < 256; ++i) {
        if (pi[i] > 0) {
            bajtoviVerovatnoce.push_back({static_cast<uint8_t>(i), pi[i]});
        }
    }
    return bajtoviVerovatnoce;
}

void insertionSortOpadajuće(vector<BajtVerovatnoca>& bajtoviVerovatnoce) {
    int n = bajtoviVerovatnoce.size();
    for (int i = 1; i < n; ++i) {
        BajtVerovatnoca tempBajt = bajtoviVerovatnoce[i];
        int j = i - 1;
        while (j >= 0 && bajtoviVerovatnoce[j].second < tempBajt.second) {
            bajtoviVerovatnoce[j + 1] = bajtoviVerovatnoce[j];
            j--;
        }
        bajtoviVerovatnoce[j + 1] = tempBajt;
    }
}
```

Glavni program (main)

U glavnom programu se pozivaju sve gore navedene funkcije, odnosno testiraju se navedene funkcije algoritama uz nastajanje izveštaja o uspesnosti i performansama kompresije.

Merenje trajanja kompresije i dekompresije vrši se pomoću šablonske funkcije **izmeriVreme**, koja prihvata proizvoljnu funkciju ili lambda izraz kao argument. Unutar nje, tačni vremenski trenuci pre i posle izvršavanja operacije beleže se pozivom `high_resolution_clock::now()`. Razlika između krajnjeg i

početnog vremena konvertuje se u milisekunde pomoću tipa `duration<double, milli>` i vraća kao `double` vrednost, čime se obezbeđuje visoka preciznost merenja za svaki algoritam pojedinačno.

```
template<typename Func>
double izmeriVreme(Func funkcija) {
    auto start = high_resolution_clock::now();
    funkcija();
    auto end = high_resolution_clock::now();
    duration<double, milli> ms = end - start;
    return ms.count();
}
```

Proces kreiranja datoteke `izvestaj.txt` započinje otvaranjem izlaznog toka `ofstream` i upisivanjem zaglavlja sa informacijama o nazivu ulaznog fajla, njegovoj veličini u megabajtima i izračunatoj bajt-entropiji. Sama tabela izveštaja formatira se pomoću C++ manipulatora `std::setw`, `std::left`, `std::fixed` i `std::setprecision`, čime se definišu fiksne širine kolona i preciznost prikaza decimalnih brojeva. Upisivanje pojedinačnih redova tabele automatizovano je preko lambda funkcije `upisiRed`, koja za svaki algoritam izračunava veličinu kompresovanog fajla pozivom `dajVelicinuMB`, računa odnos kompresije kao količnik veličine originala i kompresovanog fajla, te sve izmerene parametre i status verifikacije (`uporediFajlove`) jednoobrazno upisuje u izveštaj.

Funkcija **uporediFajlove** vrši proveru integriteta podataka i potvrđuje da li je dekompresovani fajl identičan originalnom ulazu. Otvara oba fajla u binarnom režimu (`ios::binary`) i prvo proverava da li su strimovi uspešno otvoreni. Ukoliko jesu, primenjuje standardni algoritam `std::equal` zajedno sa `istreambuf_iterator<char>` iteratorima, koji sekvencijalno, bajt po bajt, porede sadržaj obaju strimova bez učitavanja celih fajlova u memoriju. Funkcija vraća `true` samo ako se svi bajtovi u potpunosti poklapaju po vrednosti i dužini, što u izveštaju označava uspeh kompresiono-dekompresionog ciklusa.

```
bool uporediFajlove(const string& f1, const string& f2) {
    ifstream stream1(f1, ios::binary);
    ifstream stream2(f2, ios::binary);

    if (!stream1 || !stream2) return false;

    return equal(istreambuf_iterator<char>(stream1), istreambuf_iterator<char>(),
                istreambuf_iterator<char>(stream2));
}
```

Funkcija **dajVelicinuMB** izračunava veličinu zadate datoteke i izražava je u megabajtima radi lakšeg prikaza u konzoli i izveštaju. Prvo pomoću `std::filesystem::exists` proverava da li fajl uopšte postoji na navedenoj putanji kako bi sprečila greške pri pristupu. Ukoliko fajl postoji, poziva **std::filesystem::file_size** za dobijanje tačnog broja bajtova na disku, konvertuje tu vrednost u `double` i deli je sa 1024^2 , čime se bajtovi prevode u realan broj megabajta koji funkcija vraća kao rezultat.

```
double dajVelicinuMB(const string& putanja) {
    if (!fs::exists(putanja)) return 0.0;
    return static_cast<double>(fs::file_size(putanja)) / (1024.0 * 1024.0);
}
```